Status	Finished
Started	Wednesday, 28 October 2026, 11:11 PM
Completed	Wednesday, 28 October 2026, 11:20 PM
Duration	9 mins 2 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given an array of names of candidates in an election. A candidate name in the array represents a vote cast to the candidate. Print the name of candidates received Max vote. If there is tie, print a lexicographically smaller name.

Examples:

Input : votes[] = {"john", "johnny", "jackie",

"johnny", "john", "jackie",

"jamie", "jamie", "john",

"johnny", "jamie", "johnny",

"john"};

Output : John

We have four Candidates with name as 'John', 'Johnny', 'jamie', 'jackie'. The candidates John and Johny get maximum votes. Since John is alphabetically smaller, we print it. Use dictionary to solve the above problem

Sample Input:

10

John

John

Johny

Jamie

Jamie

Johny

Jack

Johny

Johny

Jackie

Sample Output:

Johny

Answer: (penalty regime: 0 %)

```
import sys
from collections import Counter

def solve():
    # Read all input from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

```

```
    # First token is the number of votes, followed by candidate names
    n = int(input_data[0])
    votes = input_data[1:n + 1]

    # Dictionary to store the vote count for each candidate
    vote_counts = {}
    for candidate in votes:
        vote_counts[candidate] = vote_counts.get(candidate, 0) + 1

    # Determine maximum votes received by any candidate
    max_votes = max(vote_counts.values())

    # Filter candidates who received the maximum votes
    winners = [candidate for candidate, count in vote_counts.items() if count == max_votes]

    # In case of a tie, select the lexicographically smaller candidate name
    winners.sort()

    # Print the winner
    print(winners[0])

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	10 John John Johny Jamie Jamie Johny Jack Johny Johny Jackie	Johny	Johny	✓
✓	6 Ida Ida Ida Kiruba Kiruba Kiruba	Ida	Ida	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Give a dictionary with value lists, sort the keys by summation of values in value list.

Input : test_dict = {'Gfg' : [6, 7, 4], 'best' : [7, 6, 5]}

Output : {'Gfg': 17, 'best': 18}

Explanation : Sorted by sum, and replaced.

Input : test_dict = {'Gfg' : [8,8], 'best' : [5,5]}

Output : {'best': 10, 'Gfg': 16}

Explanation : Sorted by sum, and replaced.

Sample Input:

2

Gfg 6 7 4

Best 7 6 5

Sample Output

Gfg 17

Best 18

For example:

Input	Result
2 Gfg 6 7 4 Best 7 6 5	Gfg 17 Best 18

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all input lines from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # First line contains the number of dictionary entries
    n = int(input_data[0].strip())

    parsed_entries = []

    # Process each dictionary entry line
    for i in range(1, n + 1):
        if i < len(input_data):
            parts = input_data[i].strip().split()
            if parts:
                key = parts[0]
                # Sum up all numerical values in the list
                total_sum = sum(map(int, parts[1:]))
                parsed_entries.append((key, total_sum))

    # Sort entries primarily by the calculated sum (ascending order)
    # Python's sort is stable, preserving insertion order on equal sums
    sorted_entries = sorted(parsed_entries, key=lambda x: x[1])

    # Print each key and its summed value
    for key, total_sum in sorted_entries:
        print(f"{key} {total_sum}")

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	2 Gfg 6 7 4 Best 7 6 5	Gfg 17 Best 18	Gfg 17 Best 18	✓
✓	2 Gfg 6 6 Best 5 5	Best 10 Gfg 12	Best 10 Gfg 12	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

A sentence is a list of words that are separated by a single space with no leading or trailing spaces. Each word consists of lowercase and uppercase English letters.

A sentence can be shuffled by appending the 1-indexed word position to each word then rearranging the words in the sentence.

For example, the sentence "This is a sentence" can be shuffled as "sentence4 a3 is2 This1" or "is2 sentence4 This1 a3".

Given a shuffled sentence s containing no more than 9 words, reconstruct and return the original sentence.

Example 1:

Input:

is2 sentence4 This1 a3

Output:

This is a sentence

Explanation: Sort the words in s to their original positions "This1 is2 a3 sentence4", then remove the numbers.

Example 2:

Input:

Myself2 Me1 I4 and3

Output:

Me Myself and I

Explanation: Sort the words in s to their original positions "Me1 Myself2 and3 I4", then remove the numbers.

Constraints:

2 <= s.length <= 200

s consists of lowercase and uppercase English letters, spaces, and digits from 1 to 9.

The number of words in s is between 1 and 9.

The words in s are separated by a single space.

s contains no leading or trailing spaces.

Answer: (penalty regime: 0 %)

```
import sys

def restoreString(s: str) -> str:
    # Split the shuffled sentence into individual word-position tokens
    words = s.split()

    # Create an array to hold the restored words in 0-indexed positions
    restored = [None] * len(words)

    for word in words:
        # Extract the trailing 1-indexed position digit
        index = int(word[-1]) - 1

        # Extract the original word by stripping off the last digit
        actual_word = word[:-1]

        # Place the word in its designated restored position
        restored[index] = actual_word

    # Join all restored words with a single space
    return " ".join(restored)

def solve():
    # Read input string from standard input
    input_data = sys.stdin.read().strip()
    if input_data:
        print(restoreString(input_data))
```

```
28
29 if __name__ == '__main__':
30     solve()
```

	Input	Expected	Got	
✓	is2 sentence4 This1 a3	This is a sentence	This is a sentence	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Incorrect

Mark 0.00 out of 1.00

Create a student dictionary for n students with the student name as key and their test mark assignment mark and lab mark as values. Do the following computations and display the result.

1.Identify the student with the highest average score

2.Identify the student who as the highest Assignment marks

3.Identify the student with the Lowest lab marks

4.Identify the student with the lowest average score

Note:

If more than one student has the same score display all the student names

Sample input:

4

James 67 89 56

Lalith 89 45 45

Ram 89 89 89

Sita 70 70 70

Sample Output:

Ram

James Ram

Lalith

Lalith

For example:

Input	Result
4 James 67 89 56 Lalith 89 45 45 Ram 89 89 89 Sita 70 70 70	Ram James Ram Lalith Lalith

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all tokens from stdin
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Extract total number of students
    n = int(input_data[0])

    students = {}
    idx = 1

    # Read each student name followed by their 3 integer marks
    for _ in range(n):
        if idx >= len(input_data):
            break
        name = input_data[idx]
```

```
        test_mark = int(input_data[idx + 1])
        assign_mark = int(input_data[idx + 2])
        lab_mark = int(input_data[idx + 3])

        students[name] = [test_mark, assign_mark, lab_mark]
        idx += 4

    if not students:
        return

    # Store exact integer total sum to avoid float comparison issues
    student_stats = []
    for name, marks in students.items():
        total_sum = sum(marks)
        student_stats.append({
            'name': name,
            'total_sum': total_sum,
            'assignment': marks[1],
            'lab': marks[2]
        })

    # 1. Identify highest average score (highest total sum)
    max_sum = max(s['total_sum'] for s in student_stats)
    highest_avg = [s['name'] for s in student_stats if s['total_sum'] == max_sum]

    # 2. Identify highest Assignment marks
    max_assign = max(s['assignment'] for s in student_stats)
    highest_assign = [s['name'] for s in student_stats if s['assignment'] == max_assign]

    # 3. Identify lowest lab marks
    min_lab = min(s['lab'] for s in student_stats)
    lowest_lab = [s['name'] for s in student_stats if s['lab'] == min_lab]

```

	Input	Expected	Got	
✓	4 James 67 89 56 Lalith 89 45 45 Ram 89 89 89 Sita 70 70 70	Ram James Ram Lalith Lalith	Ram James Ram Lalith Lalith	✓

Your code failed one or more hidden tests.

Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

A company wants to send its quotation secretly to its client. The company decided to encrypt the amount they are sending to their client with some special symbols so that the equation amount will not be revealed to any external person. They used the special symbols !,@,#,$,%,^,&,*,>,< for 0,1,2,3,4,5,6,7,8,9 respectively. Write a python code to help the company to convert the amount to special symbols.

(Value rounded off to 2 decimal points)

Input

n: Float data type which reads amount to send

Output

s: : String data type which displays symbols

Sample Testcase 1

Input

10000

Output

@!!!!.!!

Sample Testcase2

1234.56

Output

@#$%.^&

For example:

Input	Result
1345.23	@$%^.#$
15000.59	@^!!!.^<
156789	@^&*><.!!

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read input value from standard input
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Parse input as float
    amount = float(input_data)

    # Format amount rounded to 2 decimal places as a string
    formatted_amount = f"{amount:.2f}"

    # Symbol mapping for digits 0-9
    symbol_map = {
        '0': '!',
        '1': '@',
        '2': '#',
        '3': '$',
        '4': '%',
        '5': '^',
        '6': '&',
        '7': '*',
        '8': '>',
        '9': '<',
        '.': '.'  # Keep decimal point as is
    }

    # Map each character in the formatted string to its corresponding symbol
    encrypted_amount = "".join(symbol_map[char] for char in formatted_amount)

```

```
    print(encrypted_amount)

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	1345.23	@$%^.#$	@$%^.#$	✓
✓	15000.59	@^!!!.^<	@^!!!.^<	✓
✓	1234	@#$%.!!	@#$%.!!	✓
✓	156789	@^&*><.!!	@^&*><.!!	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

A sentence is a string of single-space separated words where each word consists only of lowercase letters.A word is uncommon if it appears exactly once in one of the sentences, and does not appear in the other sentence.

Given two sentences s1 and s2, return a list of all the uncommon words. You may return the answer in any order.

Example 1:

Input: s1 = "this apple is sweet", s2 = "this apple is sour"

Output: ["sweet","sour"]

Example 2:

Input: s1 = "apple apple", s2 = "banana"

Output: ["banana"]

Constraints:

1 <= s1.length, s2.length <= 200

s1 and s2 consist of lowercase English letters and spaces.

s1 and s2 do not have leading or trailing spaces.

All the words in s1 and s2 are separated by a single space.

Note:

Use dictionary to solve the problem

For example:

Input	Result
this apple is sweet this apple is sour	sweet sour

Answer: (penalty regime: 0 %)

```
import sys
from collections import Counter

def uncommonFromSentences(s1: str, s2: str) -> list[str]:
    # Combine words from both sentences
    words = s1.split() + s2.split()

    # Count frequencies of each word using a dictionary
    word_counts = Counter(words)

    # Filter words that appear exactly once overall
    return [word for word, count in word_counts.items() if count == 1]

def solve():
    # Read input lines from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    s1 = input_data[0].strip()
    s2 = input_data[1].strip() if len(input_data) > 1 else ""

    result = uncommonFromSentences(s1, s2)

    # Print space-separated result
    print(" ".join(result))

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	this apple is sweet this apple is sour	sweet sour	sweet sour	✓
✓	apple apple banana	banana	banana	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given a number, convert it into corresponding alphabet.

Input	Output
1	A
26	Z
27	AA
676	YZ

Input Format

Input is an integer

Output Format

Print the alphabets

Constraints

1 <= num <= 4294967295

Sample Input 1

26

Sample Output 1

Z

For example:

Test	Result
`print(excelNumber(26))`	Z

Answer: (penalty regime: 0 %)

Reset answer

```
import sys

def excelNumber(n: int) -> str:
    result = []

    while n > 0:
        # Decrement n by 1 to handle 1-based indexing
        n -= 1

        # Find the remainder to determine the character (0 -> 'A', 25 -> 'Z')
        remainder = n % 26
        result.append(chr(ord('A') + remainder))

        # Move to the next higher order digit
        n //= 26

    # Reverse to get the correct left-to-right order
    return "".join(reversed(result))

def solve():
    input_data = sys.stdin.read().split()
    if input_data:
        n = int(input_data[0])
        print(excelNumber(n))

if __name__ == '__main__':
    solve()
```

	Test	Expected	Got	
✓	`print(excelNumber(26))`	`Z`	`Z`	✓
✓	`print(excelNumber(27))`	`AA`	`AA`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

In the game of Scrabble™, each letter has points associated with it. The total score of a word is the sum of the scores of its letters. More common letters are worth fewer points while less common letters are worth more points. The points associated with each letter are shown below:

Points Letters

1 A, E, I, L, N, O, R, S, T and U

2 D and G

3 B, C, M and P

4 F, H, V, W and Y

5 K

8 J and X

10 Q and Z

Write a program that computes and displays the Scrabble™ score for a word. Create a dictionary that maps from letters to point values. Then use the dictionary to compute the score.

A Scrabble™ board includes some squares that multiply the value of a letter or the value of an entire word. We will ignore these squares in this exercise.

Sample Input

REC

Sample Output

REC is worth 5 points.

For example:

Input	Result
REC	REC is worth 5 points.

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read input from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    word = input_data[0].strip()

    # Map each letter to its respective Scrabble point value
    scores = {
        'A': 1, 'E': 1, 'I': 1, 'L': 1, 'N': 1, 'O': 1, 'R': 1, 'S': 1, 'T': 1, 'U': 1,
        'D': 2, 'G': 2,
        'B': 3, 'C': 3, 'M': 3, 'P': 3,
        'F': 4, 'H': 4, 'V': 4, 'W': 4, 'Y': 4,
        'K': 5,
        'J': 8, 'X': 8,
        'Q': 10, 'Z': 10
    }

    # Compute total score by summing point values of characters in the word
    total_score = sum(scores.get(letter.upper(), 0) for letter in word)

    # Print formatted output matching the sample result
    print(f"{word} is worth {total_score} points.")

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	GOD	GOD is worth 5 points.	GOD is worth 5 points.	✓
✓	REC	REC is worth 5 points.	REC is worth 5 points.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Objective:

Develop a Python program that takes an input string from the user and counts the number of occurrences of each vowel (a, e, i, o, u) in the string. The program should be case-insensitive, meaning it should treat uppercase and lowercase vowels as the same.

Description:

Vowels play a significant role in the English language and other alphabet-based languages. Counting vowels in a given string is a fundamental task that can be applied in various text processing applications, including speech recognition, linguistic research, and text analysis. The objective of this problem is to create a Python script that accurately counts and displays the number of times each vowel appears in a user-provided string.

Program Requirements:

Input:

First line reading String as input, The string can contain any characters, including letters, numbers, and special characters.

Output:

Display the number of occurrences of each vowel in the string.

The output should list each vowel followed by its count.

Example:

Consider the following example for better understanding:

- **Input:** "Python Programming"
- **Output**

  a = 1
  e = 0
  i = 1
  o = 2
  u = 0

For example:

Input	Result
Hello World	a = 0 e = 1 i = 0 o = 2 u = 0
Python	a = 0 e = 0 i = 0 o = 1 u = 0

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read input string from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    input_string = input_data[0]

    # Initialize dictionary for vowels with 0 counts
    vowels = ['a', 'e', 'i', 'o', 'u']
    vowel_counts = {vowel: 0 for vowel in vowels}

    # Convert string to lowercase and count occurrences of each vowel
    for char in input_string.lower():
        if char in vowel_counts:
            vowel_counts[char] += 1

```

```
    # Output each vowel with its count
    for vowel in vowels:
        print(f"{vowel} = {vowel_counts[vowel]}")

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	Hello World	a = 0 e = 1 i = 0 o = 2 u = 0	a = 0 e = 1 i = 0 o = 2 u = 0	✓
✓	AEIOU aeio u	a = 2 e = 2 i = 2 o = 2 u = 2	a = 2 e = 2 i = 2 o = 2 u = 2	✓
✓	Python	a = 0 e = 0 i = 0 o = 1 u = 0	a = 0 e = 0 i = 0 o = 1 u = 0	✓
✓	abcdefghijklmnopqrstuvwxyz	a = 1 e = 1 i = 1 o = 1 u = 1	a = 1 e = 1 i = 1 o = 1 u = 1	✓
✓	12345!@#$%AEIOU	a = 1 e = 1 i = 1 o = 1 u = 1	a = 1 e = 1 i = 1 o = 1 u = 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

You are given a string `word`. A letter is called **special** if it appears both in lowercase and uppercase in `word`.

Your task is to return the number of **special** letters in `word`.

Constraints

- The input string `word` will contain only alphabetic characters (both lowercase and uppercase).
- The solution must utilize a dictionary to determine the number of special letters.
- The function should handle various edge cases, such as strings without any special letters, strings with only lowercase or uppercase letters, and mixed strings.

Examples

Example 1:

Input: word = "aaAbcBC"
Output: 3

Explanation:
The special characters in `word` are 'a', 'b', and 'c'.

Example 2:

Input: word = "abc"
Output: 0

Explanation:
No character in `word` appears in uppercase.

For example:

Test	Result
`print(count_special_letters("AaBbCcDdEe"))`	5

Answer: (penalty regime: 0 %)

Reset answer

```
import sys

def count_special_letters(word: str) -> int:
    # Dictionary to track presence of lowercase and uppercase forms for each letter
    # Structure: { 'a': {'has_lower': False, 'has_upper': False}, ... }
    letter_tracker = {}

    for char in word:
        lower_char = char.lower()

        # Initialize entry if letter not seen before
        if lower_char not in letter_tracker:
            letter_tracker[lower_char] = {'has_lower': False, 'has_upper': False}

        # Check casing and update dictionary flags
        if char.islower():
            letter_tracker[lower_char]['has_lower'] = True
        elif char.isupper():
            letter_tracker[lower_char]['has_upper'] = True

    # Count how many letters have both lowercase and uppercase occurrences
    special_count = 0
    for flags in letter_tracker.values():
        if flags['has_lower'] and flags['has_upper']:
            special_count += 1

    return special_count

def solve():
    # Read input from standard input
    input_data = sys.stdin.read().split()
    if input_data:
        word = input_data[0].strip()
        print(count_special_letters(word))
```

```
35
36 if __name__ == '__main__':
37     solve()
```

	Test	Expected	Got	
✓	print(count_special_letters("AaBbCcDdEe"))	5	5	✓
✓	print(count_special_letters("ABCDE"))	0	0	✓
✓	print(count_special_letters("abcde"))	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.